

Predicting Bank Term Deposit Subscriptions Using Machine Learning: A Classification Study on the Bank Marketing Dataset

Thien Huynh (117654967) Shane Patandin (114326911)
Ishaan Reddy (116877260) Matthew Tranquada (101628633)
Prerana Somani (117325436)

Stony Brook University

April 29, 2026

Abstract

This paper studies the Bank Marketing dataset to predict whether a client will subscribe to a term deposit. Using the provided train/test split in `final.ipynb`, we build leakage-aware preprocessing and compare classical machine learning models with a PyTorch tabular transformer. A key workflow decision is to remove `duration` from both training and testing inputs because call duration is only known after the call ends and would leak the outcome into the model. The classical pipeline then engineers a prior-contact feature, caps extreme campaign counts, one-hot encodes categorical variables, and keeps scaling and SMOTE inside cross-validation pipelines when they are used. The transformer track adds richer feature engineering, learned categorical embeddings, numerical feature projections, a Transformer encoder, validation-based early stopping, and threshold selection. On the final classical test comparison, Optuna LightGBM achieved the highest balanced accuracy at 73.58%, while Optuna XGBoost achieved the highest test AUC at 78.32%. The best validation-AUC transformer was `ft_wide`, but its test balanced accuracy was close to the optional XGBoost baseline. These results show that accuracy alone is not enough for this imbalanced marketing task.

Index Terms: bank marketing, term deposit prediction, imbalanced classification, LightGBM, XGBoost, tabular transformer

I Introduction

A Context & Motivation

Every phone call in a bank marketing campaign has a cost. When a representative contacts someone who was never going to subscribe, that is wasted time, wasted budget, and often a frustrated customer. At the same time, missing a client who would have said yes is a missed opportunity. A model that can separate the two, before the calls go out, has real value.

The UCI Bank Marketing dataset, collected from out-bound telemarketing campaigns at a Portuguese bank, offers a realistic setting to study this problem [1]. It includes client demographics, prior campaign history, and macroeconomic indicators alongside a binary outcome: did the client subscribe to a term deposit or not? The dataset is widely used as a benchmark in this space, in part because it is genuinely difficult. The class imbalance is steep, several variables require careful handling to avoid inflating results, and the obvious metric (accuracy) can give a very misleading picture.

B Problem Statement

We treat this as a supervised binary classification task. Given what we know about a client before a call is placed, their demographics, their history with past campaigns, and the current economic environment, can we predict whether they will subscribe? The key complication is that only around 11% of contacts in this dataset did subscribe, so any model that just predicts “no” for everyone would already score nearly 89% accuracy. That means we need to track sensitivity (how many real subscribers we catch) alongside accuracy, not just report whichever number looks most impressive.

C Objectives

- **Data Preparation:** Audit both files, identify problematic features, and preprocess in a way that does not inflate performance.
- **Model Development:** Train and compare Logistic Regression, Random Forest, XGBoost, LightGBM, K-Nearest Neighbors, and a PyTorch tabular transformer under a leakage-aware setup.
- **Performance Evaluation:** Report accuracy, sensitivity, specificity, balanced accuracy, AUC, and threshold-dependent operating points on a held-out test set.

- **Model Selection:** Determine which model wins on raw accuracy and which one offers the better trade-off for real campaign use.

II Related Work

A Prior Studies on the Bank Marketing Dataset

The Bank Marketing dataset introduced by Moro, Cortez, and Rita has become a standard benchmark for direct-marketing response prediction [1]. Prior studies commonly compare linear baselines, tree ensembles, and boosted methods on this dataset because it mixes structured categorical data with strong class imbalance and several temporally sensitive predictors. Logistic Regression remains a useful baseline for calibrated binary classification [8], while Random Forest and gradient-boosted trees often provide stronger nonlinear fit on heterogeneous tabular data [3, 4, 5].

B Comparative Findings

Across the literature, ensemble models frequently achieve the highest raw accuracy on telemarketing datasets, but the business value of a model depends on more than accuracy alone. When subscribers are a small minority, high specificity can mask poor recovery of the positive class. This has motivated the use of resampling, class weighting, threshold tuning, and balanced metrics in imbalanced learning studies [2, 13]. In practice, the preferred model depends on whether the campaign values fewer false positives or better detection of likely subscribers.

Recent deep learning work on tabular data adapts attention mechanisms from transformer models [6] to mixed categorical and numerical features. FT-Transformer-style models represent each feature as a token and use self-attention to learn interactions among demographics, contact history, and macroeconomic variables [7]. We include this direction as an additional experiment rather than replacing tree-based baselines, since boosted trees remain very competitive on small and medium-sized tabular datasets.

C Gap & Contribution

Our work focuses on three practical issues that are easy to mishandle in this dataset:

- leakage from the post-call variable `duration`;
- strong redundancy among macroeconomic indicators;
- the temptation to judge models only by accuracy despite an 88.73% majority class.

Class	Relative bar	Count
No		29,239
Yes		3,712

Figure 1: Training-set class distribution. The positive subscription class is a small minority.

We contribute a clean comparison of five classical models under a shared preprocessing pipeline, add a transformer experiment in `final.ipynb`, use the provided testing file for final comparisons, and explicitly report the trade-off between accuracy, sensitivity, and specificity.

III Background

A Dataset Description

The dataset used in this report is the UCI Bank Marketing dataset [1], distributed here as two provided files: `training_data.csv` with 32,951 rows and `testing_data.csv` with 8,237 rows, for a total of 41,188 observations. Each row represents a bank client contact and includes demographic information, prior campaign outcomes, and macroeconomic context. The target variable `y` indicates whether the client subscribed to a term deposit.

B Attributes

The dataset comprises 21 columns: 20 predictors and one binary target.

C Class Imbalance

The dataset is strongly imbalanced and the train/test split preserves the same class ratio:

- Training set: 29,239 **no** (88.73%) and 3,712 **yes** (11.27%).
- Testing set: 7,309 **no** (88.73%) and 928 **yes** (11.27%).

A naive classifier that always predicts **no** would already reach 88.73% accuracy on the test set, so class-wise metrics are essential.

D Exploratory Data Analysis

The notebook audit surfaced several important preprocessing signals:

- No blank or null values were present in either CSV file.
- The largest explicit **unknown** bucket was **default**, affecting 6,916 training rows (20.99%).
- The **housing** and **loan** unknown flags perfectly overlapped in 804 rows.

Table 1: Dataset Attributes

#	Column	Type	Notes
1	age	int	Client age in years.
2	job	object	Job type, including categories such as admin., technician, retired.
3	marital	object	Marital status.
4	education	object	Education level.
5	default	object	Has credit in default; includes unknown .
6	housing	object	Has housing loan; includes unknown .
7	loan	object	Has personal loan; includes unknown .
8	contact	object	Contact communication type.
9	month	object	Last contact month.
10	day_of_week	object	Last contact day of week.
11	duration	int	Last contact duration in seconds; dropped because it is post-outcome.
12	campaign	int	Number of contacts during the current campaign.
13	pdays	int	Days since previous contact; 999 means never previously contacted.
14	previous	int	Number of contacts before this campaign.
15	poutcome	object	Outcome of the previous marketing campaign.
16	emp.var.rate	float	Employment variation rate.
17	cons.price.idx	float	Consumer price index.
18	cons.conf.idx	float	Consumer confidence index.
19	euribor3m	float	Euribor 3-month rate.
20	nr.employed	float	Number of employees indicator.
21	y	object	Target: yes or no .

Field	Relative bar	Unknown rate
default		20.99%
education		4.24%
housing		2.44%
loan		2.44%
job		0.82%
marital		0.19%

Figure 2: Unknown-value rates in the training data. Treating **unknown** as its own category preserves informative missingness.

Variable	Relative bar	Correlation
emp.var.rate		0.97
nr.employed		0.95
cons.price.idx		0.69
cons.conf.idx		0.28

Figure 3: Selected correlations with **euribor3m**. The strongest overlaps motivated checking redundancy among the economic variables.

- The macroeconomic variables were strongly correlated, especially **emp.var.rate** with **euribor3m** (0.97) and **nr.employed** with **euribor3m** (0.95).
- The sentinel value **pdays=999** appeared in 96.26% of training rows, motivating a derived indicator for prior contact history.
- The variable **duration** had to be removed because it is known only after the call ends and therefore leaks the target.

IV Methods

This section follows the workflow in **final.ipynb**. The notebook uses two modeling tracks: a classical machine learning track and a transformer track. They share the same raw data and the same basic leakage rule, but

they prepare the features differently because scikit-learn tree/linear models and a neural transformer expect different input formats.

A Shared Data Audit

The notebook first checks the column types, missing values, **unknown** categories, class balance, numeric summaries, and correlations among the economic variables. There were no null values in the training file, but several categorical fields used **unknown** as an explicit value. The largest was **default**, with 6,916 unknown values. The **housing** and **loan** unknown values also fully overlapped in 804 rows.

Both tracks remove **duration**. This variable records call length, so it is only known after the call has already happened. Since the project goal is to predict before making the call, **duration** cannot be used as a model input.

B Classical Workflow Preprocessing

The classical workflow starts by copying the training and testing data, dropping **duration**, and keeping the original train/test split. It also checks correlations among the economic variables. **emp.var.rate** and **nr.employed** are highly correlated with **euribor3m**, but the actual drop statements for those columns are commented out in **final.ipynb**. For that reason, those economic variables remain in the classical feature matrix. While these highly correlated parameters can affect the Logistic Regression model performance, we opted to include them since the higher-performance models (Random Forest, XGBoost, LightGBM) are all tree-based and are not affected by high correlation.

The **pdays** column uses 999 to mean that the client was not contacted in a previous campaign. To make this easier for the models to use, the notebook creates **was_contacted_before**, which is 1 when **pdays** is not 999 and 0 otherwise.

The `campaign` column is capped at the 95th percentile of the training data. In the saved notebook output, this cap is 7.0. This keeps very large campaign counts from having too much influence, especially for models that use distances or linear weights.

The target `y` is converted from `yes/no` into 1/0. The notebook then one-hot encodes all categorical predictors with `drop_first=False`, so `unknown` stays as its own category. The encoded train and test matrices are aligned so that both files have the same dummy columns. After this step, the classical feature matrix has 63 columns.

Scaling and SMOTE are not applied once to the full feature matrix. Instead, they are placed inside the relevant model pipelines. This matters because fitting a scaler or SMOTE before cross-validation would let each validation fold influence preprocessing. Keeping these steps inside the pipeline means each fold fits its own preprocessing only on that fold's training portion.

C Transformer Workflow Preprocessing

The transformer workflow reloads the raw CSV files instead of reusing the one-hot encoded classical matrices. This is important because the transformer learns embeddings for categorical fields, so it needs categorical features as category IDs rather than dummy variables.

The transformer feature helper removes `y` and `duration` from the input frame, then creates a richer set of features. For campaign activity, it uses a 99th-percentile cap fitted on the internal transformer training split, plus `campaign_capped`, `campaign_log`, and a binned `campaign_group`. For demographics, it creates `age_bin` and `education_ord`, where education levels are mapped to an ordered numeric scale and `unknown` education is filled with the median ordered value.

The transformer workflow also expands contact-history information more than the classical workflow. It creates `pdays_missing`, `pdays_clean`, `pdays_recent`, and `contacted_before`; the last one is true when either `pdays` is not 999 or `previous` is greater than zero. It also adds `previous_success`, `previous_failure`, and a binned `previous_group`.

For economic context, the transformer workflow keeps the original economic variables and adds regime bins: `emp_regime`, `euribor_regime`, `confidence_regime`, and `employment_regime`. It then adds categorical interaction features: `contact_month`, `poutcome_previous`, `pdays_previous`, `job_education`, `contact_poutcome`, and `month_emp_regime`.

After feature creation, the notebook makes an internal 80/20 stratified train-validation split with seed 580. The transformer preprocessor is fitted only on the internal training split. Categorical levels with fewer than 30 training examples are not kept as separate categories; they map to an `__unknown__` bucket. Numeric columns are standardized using means and standard deviations from the internal training split. In the saved output, this gives

23 categorical fields and 18 numerical fields for the transformer.

D Main Difference Between the Two Workflows

The classical workflow produces one flat, one-hot encoded matrix with 63 columns. It keeps feature engineering fairly simple: remove `duration`, add one prior-contact flag, cap `campaign`, encode the target, one-hot encode categories, and align train/test columns.

The transformer workflow produces two inputs: categorical ID arrays and standardized numerical arrays. It adds many more derived features, keeps categorical structure for embeddings, groups rare categories, standardizes numeric inputs from the internal training split, and uses a validation split for model and threshold selection. In short, the classical workflow is simpler and wider after one-hot encoding, while the transformer workflow is richer and token-based.

E Classical Model Screening

The classical track compares Logistic Regression, Random Forest, XGBoost, LightGBM, and K-Nearest Neighbors using 5-fold stratified cross-validation with `random_state=42`. The notebook reports balanced accuracy, accuracy, sensitivity, specificity, and an AUC-labeled score for this screening table.

Logistic Regression uses `class_weight='balanced'` with scaling. Random Forest uses SMOTE inside the pipeline. XGBoost uses SMOTE and `scale_pos_weight`. LightGBM uses native class balancing. KNN uses scaling and SMOTE.

After screening, the notebook tunes LightGBM, Random Forest, and XGBoost using balanced accuracy. LightGBM and XGBoost use `RandomizedSearchCV`; Random Forest uses `GridSearchCV`. The notebook also runs 75-trial Optuna searches for LightGBM and XGBoost. In Optuna, each trial samples a hyperparameter set, scores it with stratified 5-fold cross-validation balanced accuracy, and the TPE sampler uses earlier trials to suggest stronger settings. This lets us choose the model configuration with the best cross-validated balanced accuracy before fitting it on the full training set and evaluating it on the held-out test set.

Listing 1. Representative Tree-Based Pipeline

```
Pipeline([
    ('smote', SMOTE(random_state=42)),
    ('model', xgb.XGBClassifier(
        eval_metric='logloss',
        tree_method='hist',
        device=xgb_device,
        random_state=42,
        n_jobs=-1,
        scale_pos_weight=scale_pos
```

```

    ))
])

```

```

cls = self.cls.expand(x.shape[0], -1, -1)
x = torch.cat([cls, x], dim=1)
encoded = self.encoder(x)
return self.head(encoded[:, 0]).squeeze(1)

```

F PyTorch Tabular Transformer

The transformer track uses a PyTorch tabular transformer. The training set is split into an internal 80/20 stratified train-validation split using seed 580. The validation split is used for early stopping, model selection, and threshold selection.

The model maps each categorical feature to a learned embedding. Each numerical feature is projected into the same token space with a small linear layer and layer normalization. A learned [CLS] token is added before the feature tokens, passed through a Transformer encoder, and then used by the prediction head.

Listing 2. Core Transformer Structure

```

class FTTransformer(nn.Module):
    def __init__(self, cat_cardinalities, n_num,
                 d_token=48, n_heads=4,
                 n_layers=3, dropout=0.15):
        super().__init__()
        self.cat_embeddings = nn.ModuleList([
            nn.Embedding(cardinality, d_token)
            for cardinality in cat_cardinalities
        ])
        self.num_projections = nn.ModuleList([
            nn.Sequential(
                nn.Linear(1, d_token),
                nn.LayerNorm(d_token)
            )
            for _ in range(n_num)
        ])
        self.cls = nn.Parameter(
            torch.zeros(1, 1, d_token)
        )
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_token,
            nhead=n_heads,
            dim_feedforward=d_token * 4,
            dropout=dropout,
            activation='gelu',
            batch_first=True,
            norm_first=True
        )
        self.encoder = nn.TransformerEncoder(
            encoder_layer,
            num_layers=n_layers
        )
        self.head = nn.Sequential(
            nn.LayerNorm(d_token),
            nn.Linear(d_token, d_token),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(d_token, 1)
        )

    def forward(self, x_cat, x_num):
        tokens = []
        for i, emb in enumerate(self.cat_embeddings):
            tokens.append(emb(x_cat[:, i]))
        for i, proj in enumerate(self.num_projections):
            value = x_num[:, i : i + 1]
            tokens.append(proj(value))
        x = torch.stack(tokens, dim=1)

```

The notebook trains three transformer configurations: `ft_small`, `ft_medium`, and `ft_wide`. Training uses AdamW, dropout, gradient clipping, mixed precision when CUDA is available, and `BCEWithLogitsLoss` with a positive-class weight. In the saved output, `ft_wide` is selected by validation AUC, with 64-dimensional tokens, 8 attention heads, and 3 transformer layers. The 64-token and 8-head choice came from this validation comparison: 64 is divisible by 8, giving each attention head 8 dimensions, and the shared batch size of 512 gave stable updates while keeping training practical. The detailed training, testing, and threshold-selection strategy is described in the next section.

V Strategy

A Unified Modeling Pipeline

The project follows the two-track modeling strategy in `final.ipynb`:

- a **classical machine learning track** with linear, distance-based, bagging, and boosting models;
- a **PyTorch transformer track** for learning feature interactions from embedded categorical and numerical tokens.

Both tracks remove `duration`. The provided training file is the only source used for fitting and tuning models. The provided testing file is held out from training and used for final evaluation of the classical models, transformer models, and optional XGBoost baseline.

B Classical Modeling Strategy

The classical track begins with 5-fold stratified cross-validation on the training file. Logistic Regression and KNN are included as baseline checks. Random Forest, XGBoost, and LightGBM are then tuned because they are stronger fits for this tabular, nonlinear, imbalanced problem.

The notebook uses different imbalance handling by model:

- Logistic Regression uses class weighting and scaling.
- KNN uses scaling and SMOTE inside the pipeline.
- Random Forest uses SMOTE inside the pipeline.
- XGBoost uses class weighting through `scale_pos_weight`; the randomized-search XGBoost pipeline also uses SMOTE.

- LightGBM uses native imbalance handling.

Balanced accuracy is used for tuning. LightGBM and XGBoost are tuned with `RandomizedSearchCV`, Random Forest is tuned with `GridSearchCV`, and the notebook also runs 75-trial Optuna searches for LightGBM and XGBoost. Optuna helps choose the final boosted model by searching many hyperparameter combinations and keeping the one with the highest cross-validated balanced accuracy. After tuning, the selected classical models are refit on the full training file and evaluated on the held-out testing file.

C Transformer Modeling Strategy

The transformer track starts again from the raw CSV files, builds additional interaction features, and splits the provided training file into an internal training and validation split. The internal training split is used for fitting, while the validation split is used for early stopping, model selection, and threshold selection.

The main design choices are:

- categorical features are mapped to learned embeddings;
- numerical features are projected into the same token space;
- a learned [CLS] token is used as the final representation for classification;
- AdamW, dropout, gradient clipping, and weighted binary cross-entropy are used during training.

Three sizes are evaluated: `ft_small`, `ft_medium`, and `ft_wide`. In the saved notebook output, the run used CPU, and `ft_wide` was selected by validation AUC (0.7993). The optional XGBoost baseline in this track uses the same internal training/validation split and is tested on the same held-out testing file.

D Threshold Optimization Strategy

Instead of relying on the default 0.5 classification threshold, we explicitly optimized decision thresholds using validation data.

Two objective functions were used:

Harmonic score:

$$H = \frac{3}{(1/\text{Accuracy}) + (1/\text{Sensitivity}) + (1/\text{Specificity})}$$

Business score:

$$B = 0.4(\text{Accuracy}) + 0.4(\text{Sensitivity}) + 0.2(\text{Specificity})$$

Here H means the harmonic score and B means the business score. H was used for balanced performance across all three metrics; B was used when we wanted more emphasis on detecting subscribers. The 0.4/0.4/0.2 business

weights give equal importance to accuracy and sensitivity, with a smaller but nonzero weight on specificity.

Thresholds were selected on validation data only and applied once to the test set to avoid bias.

E Final Evaluation Design

The final report follows the notebook outputs and separates two comparisons:

- The final classical test results for tuned and Optuna-selected models;
- The selected transformer and optional XGBoost operating points from the validation-threshold experiment.

VI Results

A Overview of Final Model Performance

The final notebook reports two sets of results. The first set is the classical-model test comparison. The second set is the transformer and optional XGBoost threshold comparison. Because the positive class is only about 11%, balanced accuracy, sensitivity, and specificity are more informative than accuracy alone.

B Classical Model Test Results

Part B reports the final held-out test results for the classical models only. Table 2 summarizes the tuned and Optuna-selected tree models after training and tuning on the provided training data.

C Classical Model Interpretation

The classical models worked well because the dataset is structured tabular data (although it is imbalanced) with many categorical groups, campaign-history fields, and economic indicators. Tree-based models are a good fit for this setting because they split clients into smaller decision segments instead of assuming one straight-line relationship. For example, they can combine information such as prior contact history, month, contact type, campaign count, age, and macroeconomic conditions when estimating subscription probability.

The boosted models, especially LightGBM and XGBoost, performed best because each new tree focuses on correcting mistakes from earlier trees. Their imbalance handling also helped the models pay more attention to the small **yes** class. Random Forest was more conservative: averaging many trees helped specificity and overall stability, but it recovered fewer subscribers than boosted models. Logistic Regression and KNN were useful baselines, but they were less able to capture the nonlinear

Model	Accuracy	Sensitivity	Specificity	Balanced Acc.	Test AUC
Optuna LightGBM (Bal)	84.08%	60.02%	87.14%	73.58%	78.18%
Optuna XGBoost (Bal)	83.88%	59.91%	86.92%	73.42%	78.32%
Tuned LightGBM (Bal)	83.84%	59.48%	86.93%	73.21%	78.18%
Tuned Random Forest (Bal)	86.03%	55.60%	89.89%	72.75%	77.64%
Tuned XGBoost (Bal)	80.17%	62.61%	82.41%	72.51%	77.26%

Table 2: Classical model performance on the test set from `final.ipynb`.

feature interactions in this dataset. We opted to use balanced accuracy as our main evaluation metric since it provides a better way for us to understand how the model performs on the unbalanced data.

D Transformer and Optional XGBoost Results

Part D is a separate threshold-based comparison for the transformer track. Table 3 reports the selected transformer operating points and the optional XGBoost baseline, all evaluated on the same held-out testing file.

E Key Observations

1. Best classical trade-off: Optuna LightGBM achieved the highest classical balanced accuracy at 73.58%. Optuna XGBoost was very close at 73.42% and had the highest classical test AUC at 78.32%.

2. Highest sensitivity: Tuned XGBoost had the highest sensitivity among the final classical models at 62.61%, but this came with lower accuracy and specificity.

3. Transformer performance: `ft.wide` was selected by validation AUC, but the highest transformer test balanced accuracy in the saved output came from `ft.small` at the default threshold. The transformer and optional XGBoost baseline were close to each other, with balanced accuracy around 72.6–73.0%.

4. Accuracy can be misleading: Some accuracy-threshold operating points reached about 90% accuracy, but their sensitivity dropped sharply. For example, the `ft.wide` accuracy threshold had 90.28% accuracy but only 22.95% sensitivity in the notebook output.

5. AUC values are close: The final classical AUC values are all in a narrow range from 77.26% to 78.32%. This means model choice depends heavily on the preferred sensitivity-specificity trade-off, not only on ranking ability.

F Notebook Caveats

The notebook has a few caveats that matter for interpretation. First, the cross-validation table labels one metric as AUC, but the scorer is built from `roc_auc_score` on class predictions rather than predicted probabilities. The final test AUC values are more meaningful because they use `predict_proba`.

Second, many models and thresholds are compared on the test file. The test data is not used for fitting, but repeated comparison can still make final performance look more optimistic than a single locked model evaluation.

Third, the active data paths in `final.ipynb` use `./training_data.csv` and `./testing_data.csv`. A local run from this repository needs either those files copied to the notebook directory or the commented `data/` paths restored.

G Summary of Findings

The results highlight three important conclusions:

- Optuna LightGBM was the strongest final classical model by balanced accuracy.
- Tuned XGBoost found the most subscribers, but at the cost of more false positives.
- Transformer models were competitive, but they did not clearly beat the best classical models.

VII Conclusion

This study evaluated classical machine learning models and a PyTorch tabular transformer for predicting term deposit subscriptions using the Bank Marketing dataset under a leakage-aware pipeline.

Using the results saved in `final.ipynb`, the best final classical model by balanced accuracy was Optuna LightGBM at 73.58%. Optuna XGBoost was almost tied on balanced accuracy and had the best AUC at 78.32%. Tuned XGBoost had the highest sensitivity, which means it found the most subscribers, but it also made more false-positive predictions.

The transformer results were close but not clearly better than the strongest classical models. The `ft.wide` model had the best validation AUC among transformer configurations, while `ft.small` at the default threshold had the highest transformer test balanced accuracy in the saved output.

Overall, this work highlights three key takeaways:

- Accuracy alone is not a reliable metric for imbalanced classification problems.
- Different models optimize different trade-offs between false positives and false negatives.

Model	Operating Point	Threshold	Accuracy	Sensitivity	Specificity	Balanced Acc.	Test AUC
FT-Small	Default 0.50	0.500	84.44%	58.19%	87.77%	72.98%	77.04%
FT-Wide	Harmonic	0.444	83.56%	58.94%	86.69%	72.82%	77.61%
FT-Medium	Business	0.540	84.01%	58.19%	87.29%	72.74%	77.44%
FT-Wide	Default 0.50	0.500	84.86%	56.79%	88.43%	72.61%	77.61%
FT-Wide	Business	0.470	84.23%	57.54%	87.62%	72.58%	77.61%
XGBoost Baseline	Business/Harmonic	0.298	84.22%	57.54%	87.60%	72.57%	77.43%

Table 3: Selected transformer and optional XGBoost operating points from `final.ipynb`.

- Tree-based models remain very competitive for this imbalance dataset.

Future work should focus on cleaner reproducibility, probability calibration, cost-sensitive threshold selection, and a final locked evaluation after choosing one model and one threshold.

Acknowledgments

The authors would like to thank Professor Weihao Wang and the teaching assistants for their guidance and support throughout this project.

VIII Group Member Contributions

The project was made possible by the following team efforts:

- **Data audit, preprocessing, and modeling:** Thien Huynh , Shane Patandin, Ishaan Reddy.
- **Literature review and metric interpretation:** Thien Huynh , Shane Patandin, Ishaan Reddy, Pre-rana Somani.
- **Report writing, editing, and final notebook/report alignment:** Thien Huynh , Shane Patandin, Ishaan Reddy, Matthew Tranquada, Pre-rana Somani.

References

- [1] S. Moro, P. Cortez, and P. Rita, “A data-driven approach to predict the success of bank telemarketing,” *Decision Support Systems*, vol. 62, pp. 22–31, 2014.
- [2] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic Minority Over-sampling Technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [3] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, pp. 5–32, 2001.
- [4] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.
- [5] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [6] A. Vaswani *et al.*, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [7] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko, “Revisiting Deep Learning Models for Tabular Data,” in *Advances in Neural Information Processing Systems*, vol. 34, 2021.
- [8] D. W. Hosmer Jr. and S. Lemeshow, *Applied Logistic Regression*. Wiley-Interscience, 2000.
- [9] T. M. Cover and P. E. Hart, “Nearest Neighbor Pattern Classification,” *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.
- [10] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [11] R. Kohavi, “A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection,” in *Proc. 14th International Joint Conference on Artificial Intelligence*, pp. 1137–1143, 1995.
- [12] J. Bergstra and Y. Bengio, “Random Search for Hyper-Parameter Optimization,” *Journal of Machine Learning Research*, vol. 13, pp. 281–305, 2012.
- [13] H. He and E. A. Garcia, “Learning from Imbalanced Data,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.

A Python Implementation

The analysis summarized in this report is implemented in the accompanying notebook `final.ipynb`. The appendix block below lists the main reproducibility files.

Listing 3. Bank Marketing Source Files

```
Notebook:
final.ipynb

Primary data files:
data/training_data.csv
data/testing_data.csv

Transformer files:
transformer_outputs/ft_transformer_bank_model.pt
transformer_outputs/transformer_test_metrics.csv
transformer_outputs/transformer_test_predictions.csv
```